



End-to-End Navigation I

Graduate Course INTR-6000P

Week 12 - Lecture 23

Changhao Chen

Assistant Professor

HKUST (GZ)

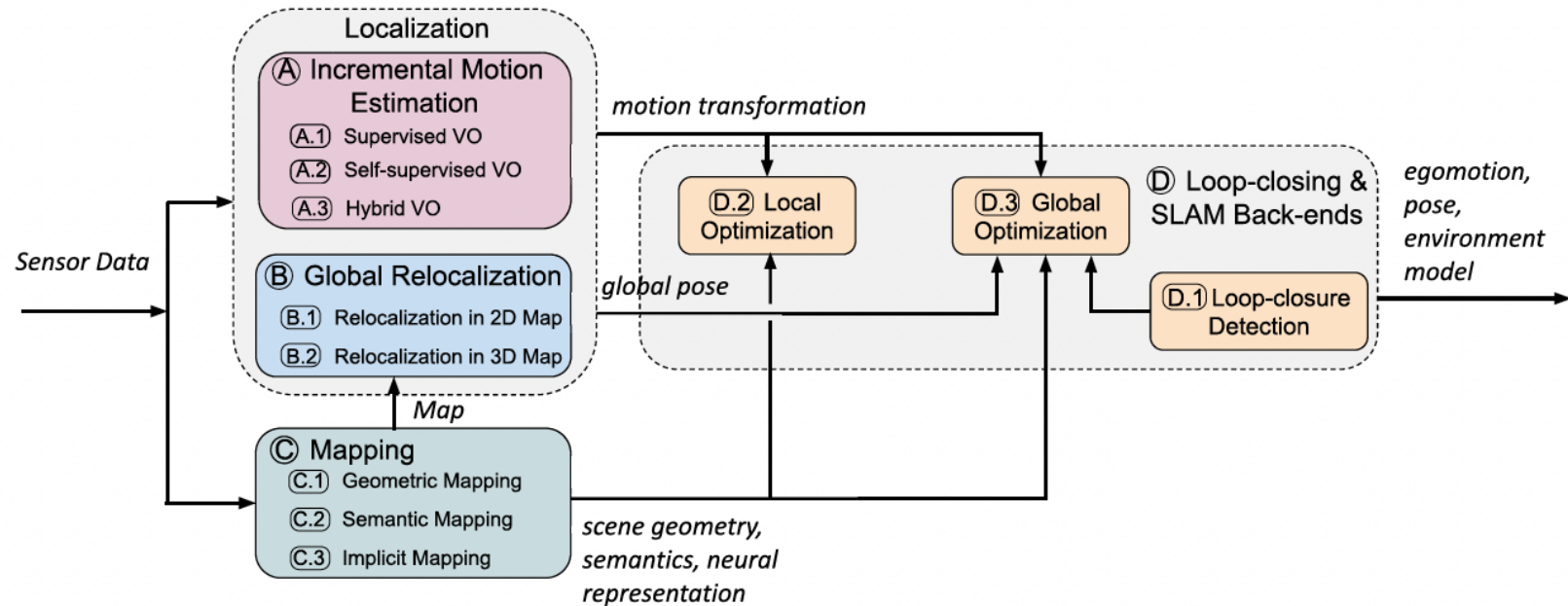
Recap: Deep Learning in SLAM

Challenges of classical SLAM:

- Feature sparsity in low-texture/low-light environments
- Scale ambiguity in monocular SLAM
- Difficulty in robust relocalization

Deep learning opportunities:

- End-to-end pose regression
- Learned features robust to illumination changes
- Semantic understanding for scene consistency



Recap: End-to-End Pose Estimation

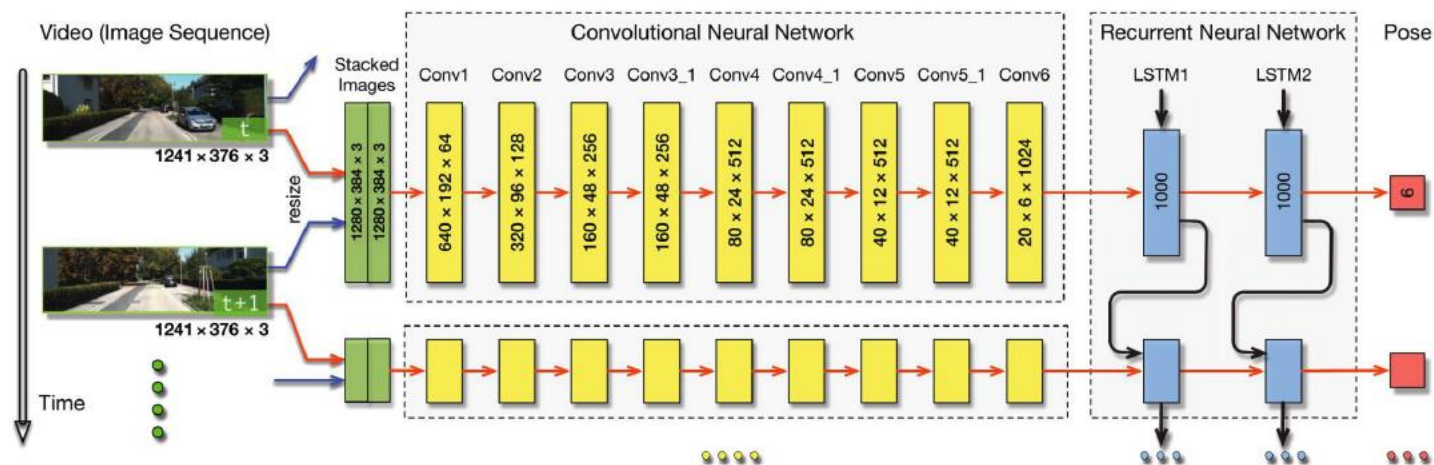
Problem: Predict relative camera pose directly from images

Methods:

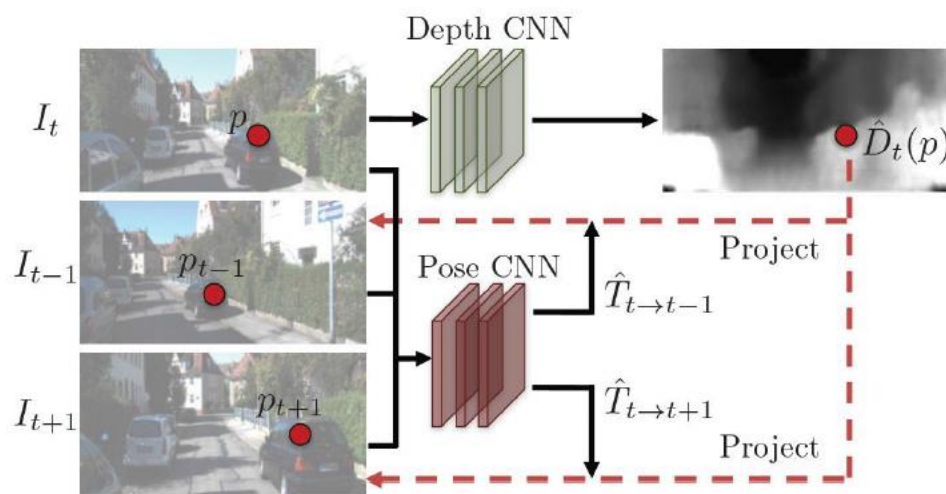
- Self-Supervised VO
- Supervised VO
- Hybrid VO

Input: Single/multiple frames or sequences

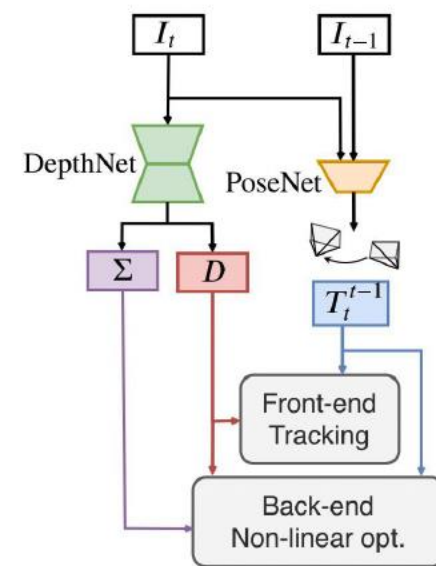
Output: 6-DoF pose



(a)



(b)



(c)

Recap: Learning based Camera Relocalization

End-to-End Camera Relocalization

PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization (2015)

- CNN architecture
- Regression losses

$$\mathbf{p} = [\mathbf{x}, \mathbf{q}]$$

- Quaternion vs Euler
- Scale balancing
- Geometric consistency losses.

$$loss(I) = \|\hat{\mathbf{x}} - \mathbf{x}\|_2 + \beta \left\| \hat{\mathbf{q}} - \frac{\mathbf{q}}{\|\mathbf{q}\|} \right\|_2$$

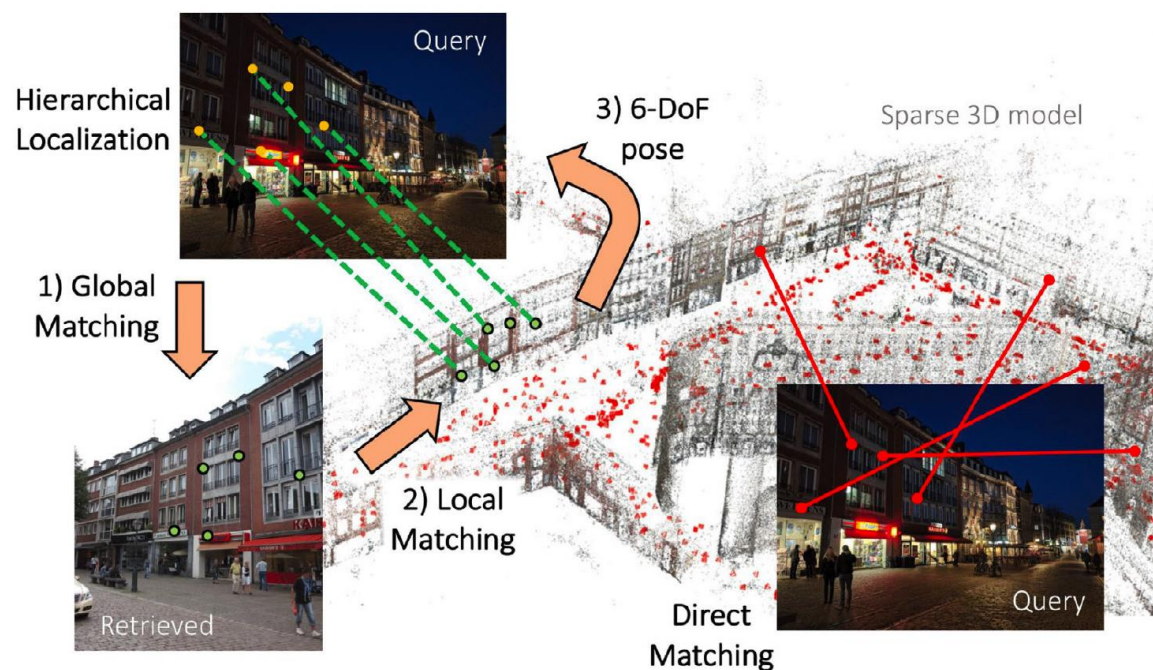
Figure 5: **7 Scenes dataset** example images from left to right; Chess, Fire, Heads, Office, Pumpkin, Red Kitchen and Stairs.

Scene	# Frames		Spatial Extent (m)	SCoRe Forest (Uses RGB-D)	Dist. to Conv. Nearest Neighbour	PoseNet	Dense PoseNet
	Train	Test					
King's College	1220	343	140 x 40m	N/A	3.34m, 2.96°	1.92m, 2.70°	1.66m, 2.43°
Street	3015	2923	500 x 100m	N/A	1.95m, 4.51°	3.67m, 3.25°	2.96m, 3.00°
Old Hospital	895	182	50 x 40m	N/A	5.38m, 4.51°	2.31m, 2.69°	2.62m, 2.45°
Shop Façade	231	103	35 x 25m	N/A	2.10m, 5.20°	1.46m, 4.04°	1.41m, 3.59°
St Mary's Church	1487	530	80 x 60m	N/A	4.48m, 5.65°	2.65m, 4.24°	2.45m, 3.98°
Chess	4000	2000	3 x 2 x 1m	0.03m, 0.66°	0.41m, 5.60°	0.32m, 4.06°	0.32m, 3.30°
Fire	2000	2000	2.5 x 1 x 1m	0.05m, 1.50°	0.54m, 7.77°	0.47m, 7.33°	0.47m, 7.02°
Heads	1000	1000	2 x 0.5 x 1m	0.06m, 5.50°	0.28m, 7.00°	0.29m, 6.00°	0.30m, 6.09°
Office	6000	4000	2.5 x 2 x 1.5m	0.04m, 0.78°	0.49m, 6.02°	0.48m, 3.84°	0.48m, 3.62°
Pumpkin	4000	2000	2.5 x 2 x 1m	0.04m, 0.68°	0.58m, 6.08°	0.47m, 4.21°	0.49m, 4.06°
Red Kitchen	7000	5000	4 x 3 x 1.5m	0.04m, 0.76°	0.58m, 5.65°	0.59m, 4.32°	0.58m, 4.17°
Stairs	2000	1000	2.5 x 2 x 1.5m	0.32m, 1.32°	0.56m, 7.71°	0.47m, 6.93°	0.48m, 6.54°

Recap: Learning based Camera Relocalization

Map based Camera Relocalization

From coarse to fine: Robust hierarchical localization at large scale (2019)



HFNet, a hierarchical localization approach based on a monolithic CNN that simultaneously predicts local features and global descriptors for accurate 6-DoF localization. We exploit the coarse-to-fine localization paradigm: we first perform a global retrieval to obtain location hypotheses and only later match local features within those candidate places

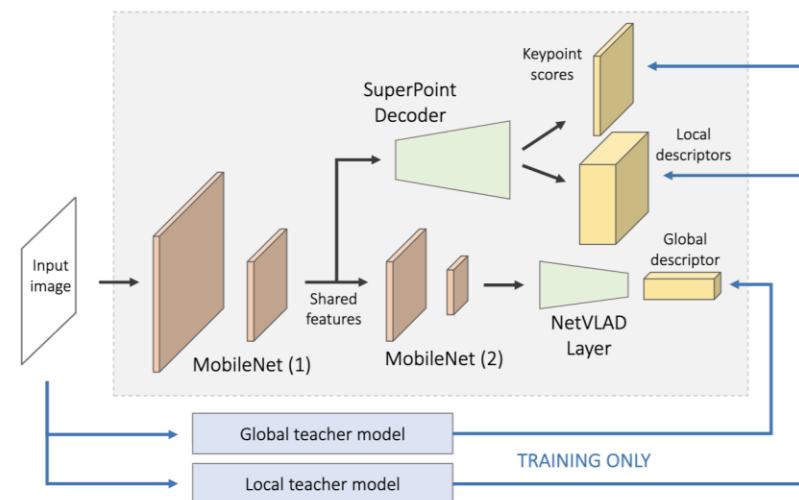
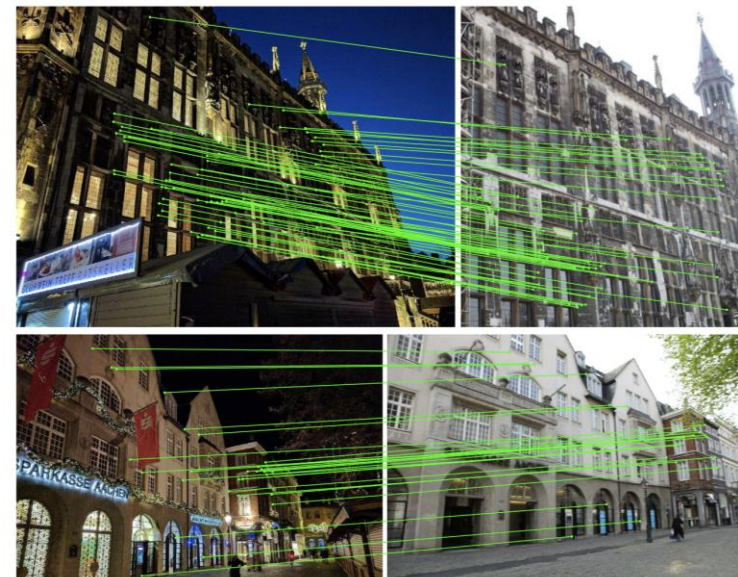


Figure 3. **HF-Net** generates three outputs from a single image: a global descriptor, a map of keypoint detection scores, and dense keypoint descriptors. All three heads are trained jointly with multi-task distillation from different teacher networks.



Recap: Semantic SLAM

- Combine geometry + semantics
- Tasks: object detection, segmentation, dynamic object handling

Benefits of Semantic SLAM

- Improved tracking and mapping
 - Scene understanding for navigation
 - Robustness to dynamic objects
-
- **Through SLAM to improve semantic recognition.**
 - **Through semantic recognition to improve SLAM.**

Recap: Summary of Semantic SLAM

Semantic SLAM enhances SLAM by integrating object-level semantic information into the mapping and localization process. Instead of representing the world with just geometric primitives (points, lines, planes), it creates a map populated with recognizable, meaningful objects (e.g., chairs, cars, doors).

- **Robustness and Accuracy:** Use the stable, recognizable nature of objects to reduce drift in localization and enable more meaningful data association across different viewpoints.
- **High-Level Understanding:** Move from a "dense but dumb" 3D point cloud to a sparse but semantically rich map that robots can reason about (e.g., "navigate to the desk").
- **Human-Robot Interaction:** Enable robots to understand and execute commands based on object semantics (e.g., "go to the refrigerator").

Paradigms and Evolution

- **Early/Object-Based:** Represents the world as a collection of semantic objects (cubes, ellipsoids). Sparse but efficient.
- **Dense Semantic SLAM:** Creates dense 3D maps where every point/voxel has a semantic label. Computationally heavy but very detailed.
- **Probabilistic Frameworks:** Modern approaches (like the one in your previous query) use factor graphs to model the uncertainty in both geometry and semantics, leading to more robust systems.

Traditional Robotics Navigation

Pipeline:

Perception → Mapping → Planning → Control

Components

- SLAM / VIO (pose estimation)
- Map building (grid / mesh / semantic)
- Motion planning (A*, RRT*)
- Low-level control (PID, MPC)

Pros

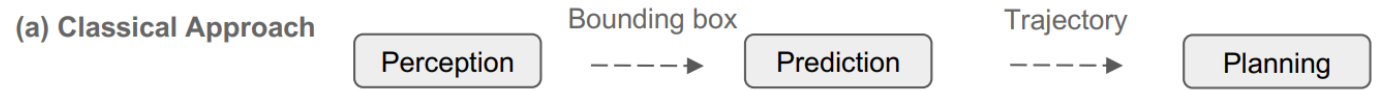
- Interpretable, stable, safe

Cons

- Complex tuning, brittle to failures
- Error accumulation across modules

Why End-to-End?

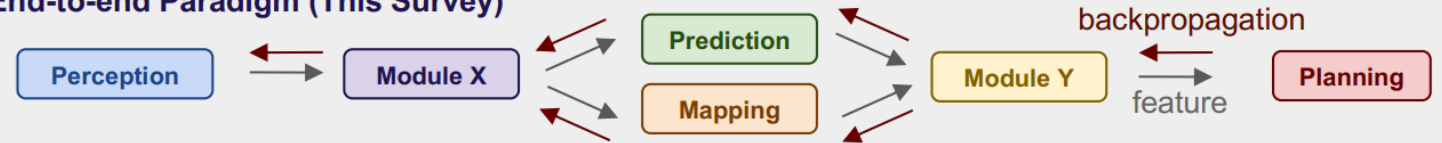
Traditional Modular Navigation:



VS.

End-to-End Navigation:

(b) End-to-end Paradigm (This Survey)



Advantages:

1. Simplicity in combining perception, prediction, and planning
2. Optimization of system and intermediate representations
3. Computational efficiency through shared backbones
4. Data-driven optimization via resource scaling

End-to-End Navigation

Definition

Direct mapping from raw sensory input to navigation actions

Inputs

- Camera, LiDAR, IMU, GPS
- Language (VLN tasks)

Outputs

- Low-level control (steering, throttle)
- Waypoints / future trajectory
- Affordances (traffic lights, drivable area)



Modality	Example
RGB cameras	end-to-end perception
LiDAR	accurate depth, 3D space
IMU	motion cues
GPS	global grounding

End-to-End Navigation

Data Flow

Input:

Visual
Sensors



HD Maps



Navigation
Signal



Vehicle
States



Language
Instruction



Output:

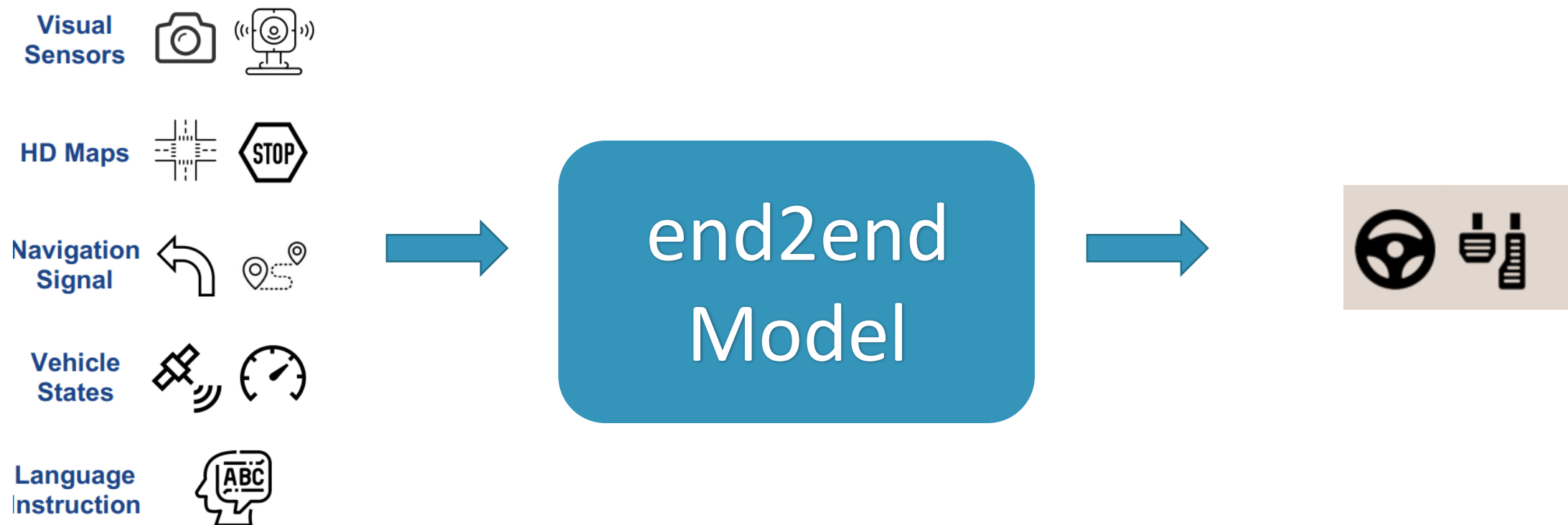
The model generates low-level control actions specific to the platform:

For Self-Driving Cars: Continuous control signals, including steering, throttle, and braking.

For Mobile Robots: Kinematic commands, typically expressed as target velocities and angular velocities.



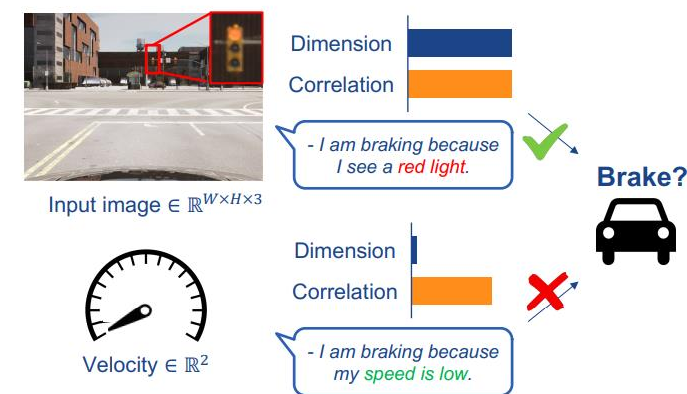
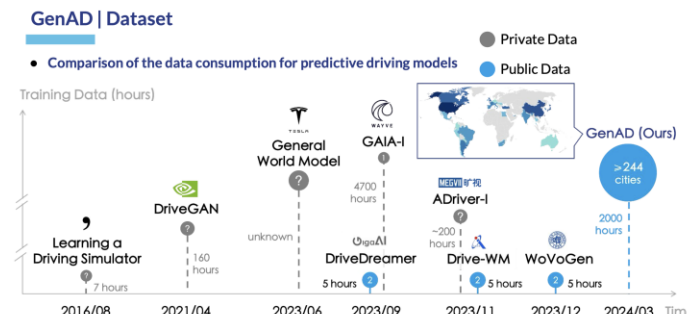
End-to-End Navigation



Output Type	Examples
Low-level control	steering, throttle
Waypoints	x,y future points
Trajectory	discrete future states
Affordances	stop/go, lane offset

Key Challenges

- **Massive Data Dependency**
Requires immense volumes of diverse and expensive labeled training data.
- **The "Black Box" Problem**
Lack of interpretability and reasoning behind model decisions.
- **Safety and Verification Hurdles**
Difficulty in certifying and ensuring reliability in all edge cases.
- **Generalization Limits**
Performance degradation in unseen or highly dynamic environments.



Challenges

Section 4

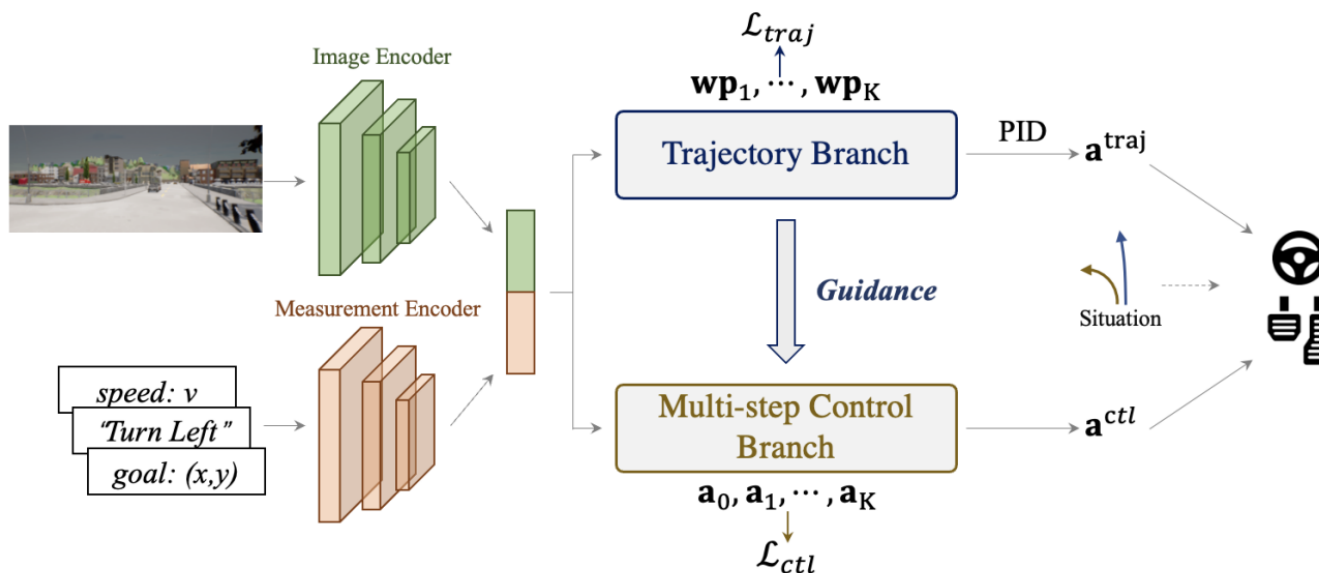


Three Typical End-to-End Modes

1) Direct Perception-to-Control

A neural policy consumes raw sensory observations (e.g., RGB images, lidar, proprioception) and **directly predicts low-level control actions**:

No intermediate maps, no planning module.



Pipeline

Camera $\rightarrow$ CNN/Transformer $\rightarrow$ actions

$$a_t = f(o_t)$$

Training

- Imitation learning (behavior cloning)
- Reinforcement Learning
- Self-supervision (rare)

Source: P. Wu, X. Jia, L. Chen, J. Yan, H. Li, and Y. Qiao, "Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline," in NeurIPS, 2022.

Three Typical End-to-End Modes

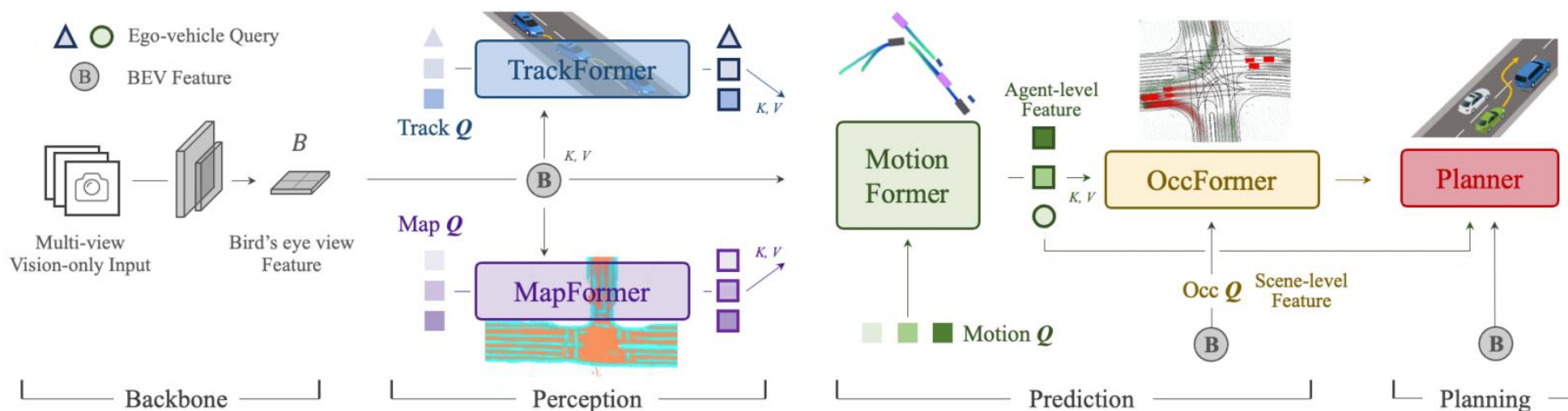
2) Perception-to-Intermediate Representation-to-Control

Perception network extracts **spatial + temporal latent representation** (learned or interpretable), then a policy predicts a **trajectory** or **waypoints**, then a controller converts these to commands.

$$o_t \rightarrow z_t \setminus (BEV / occupancy / waypoints) \rightarrow a_t$$

Training

- Imitation learning + self-supervised depth/ego-motion
- Auxiliary losses: segmentation, box prediction, map prediction



Three Typical End-to-End Modes

2) Perception-to-Intermediate Representation-to-Control

Types of Intermediate Representations

Category	Representation
Spatial	BEV map, occupancy grid, lane map
Geometric	Waypoints, 3D ego-trajectory
Agent Modeling	Driver intents, object motion
Hybrid	Affordances (e.g., lane position, TTC)

Strengths

1. Much more stable & interpretable than direct E2E
2. Compatible with rule-based safety layers
3. Better generalization across environments
4. Temporal reasoning possible

Weaknesses

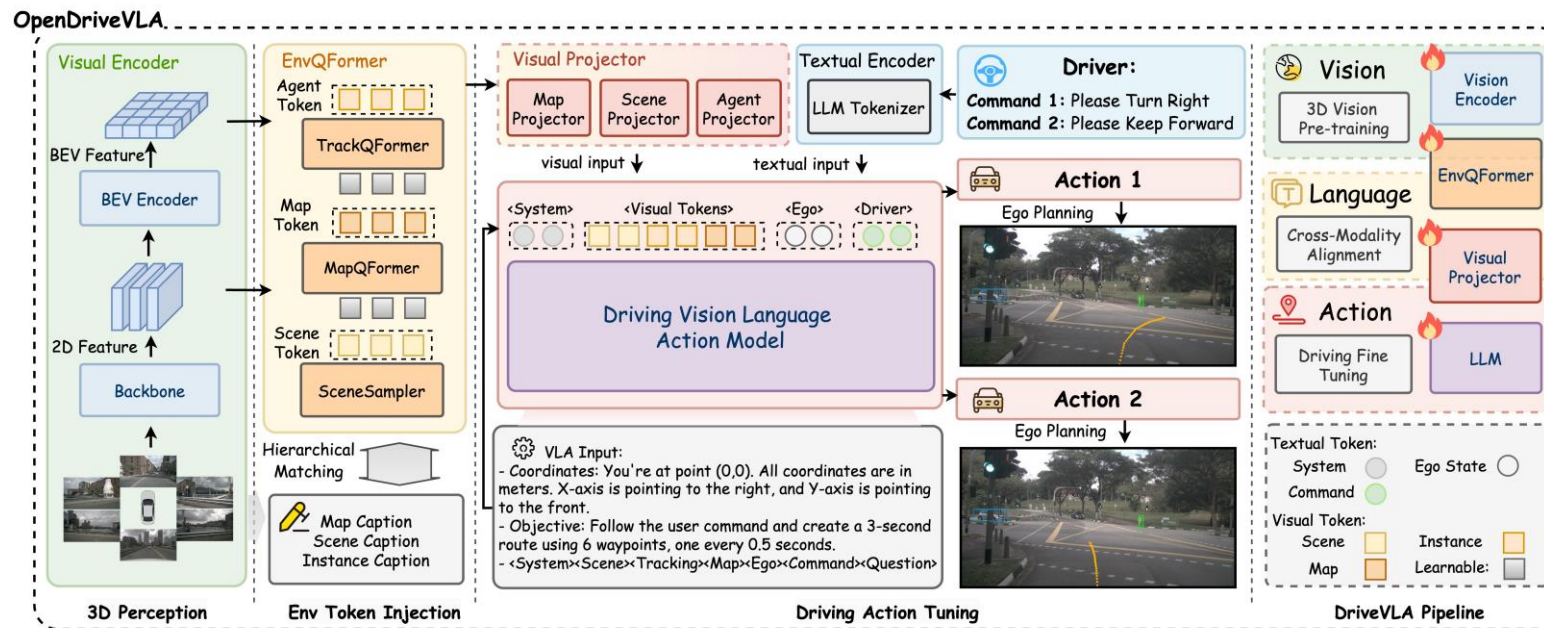
1. Training complexity
2. Requires representation design
3. Can still fail without explicit planning safety

Three Typical End-to-End Modes

3) Perception-to-High-Level Semantics-to-Decision

System extracts **explicit scene semantics** (objects, traffic signals, road topology, intents), then uses rule-based or symbolic reasoning for planning.

$$o_t \rightarrow \text{semantic_scene} \rightarrow \text{high_level_policy} \rightarrow \text{controller}$$



Semantic Outputs

- Traffic light state
- Lane & road graph
- Pedestrian/cyclist detection
- Vehicle type & behavior
- Stop signs, speed limits
- Goals, sub-goals, navigation graph

Decision Layer

- Planning graph
- Behavior prediction
- Scenario recognition (merging, overtaking)
- Logic + RL hybrid

Three Typical End-to-End Modes

3) Perception-to-High-Level Semantics-to-Decision

Strengths

1. Most interpretable
2. Best for regulatory / safety-critical deployment
3. Human-understandable decisions
4. Enables fail-safe logic (HD maps, prior knowledge)

Weaknesses

1. Highest engineering effort
2. Semantic errors → cascading failures
3. Requires large labeled datasets

Usage Examples

- **Waymo / Cruise / Mobileye** industrial stacks
- **OpenPilot + Nissan ProPILOT Assist**
- Research: semantic route planning, VLN + symbolic planners

When to Use

- Safety-critical robotics (AVs, delivery robots)
- Systems requiring explainability & compliance
- Navigation with language commands

Representations for Navigation

Three design choices :

1.Raw egocentric images

2.Intermediate features

3.Top-down Bird's-Eye-View (BEV)

Core idea

Navigation : **spatial reasoning** — BEV is spatially aligned.

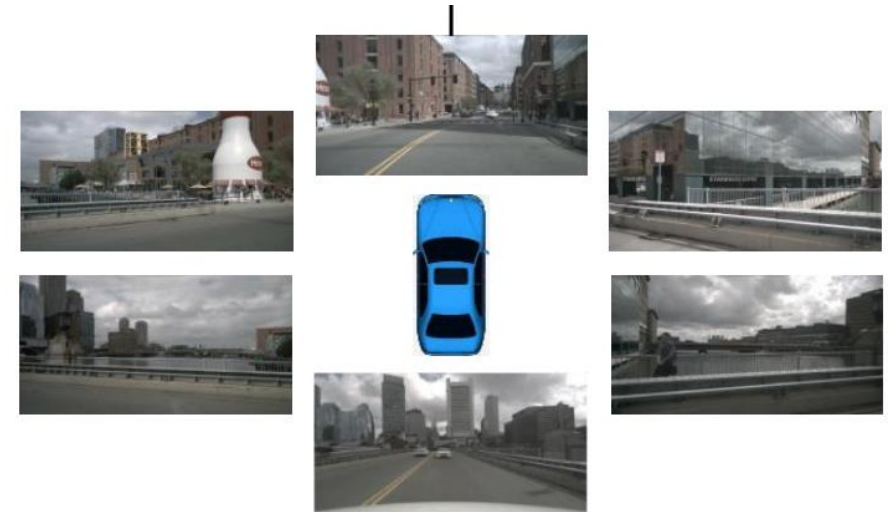
Bird's-Eye-View (BEV) Representation

Definition

Top-down feature representation of environment

Captures

- Lane lines
- Drivable area
- Obstacles
- Traffic agents
- Free space & occupancy



Multi-view Input at Time t

Bird's-Eye-View (BEV) Representation

1) Unified Spatial Representation

Transforms multi-view camera feeds into a single, coherent top-down map of the environment.

2) Core Geometric Foundation

Provides critical 3D spatial understanding, enabling precise distance measurement and object localization.

Advantages

1) Inherent Geometric Consistency

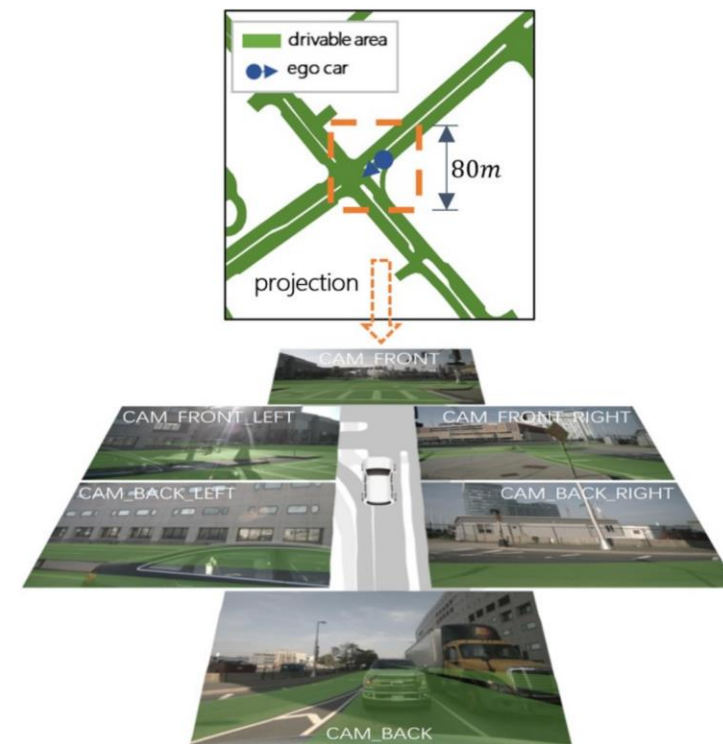
Maintains accurate spatial relationships across all perception inputs.

2) Native Multi-Sensor Fusion

Provides a unified canvas for camera, LiDAR, and radar data.

3) Intuitive Downstream Planning

Delivers an interpretable scene representation for robust decision-making.



BEV Generation Methods

From sensors to BEV:

- LiDAR projection
- Depth estimation → point cloud → grid
- Inverse perspective mapping (IPM) from camera

1) Geometric Projection

Inverse Perspective Mapping (IPM):

Transforms image pixels to the BEV space using known camera geometry.

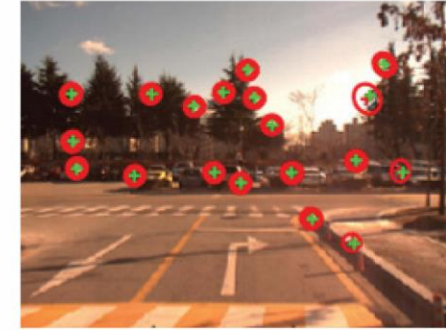
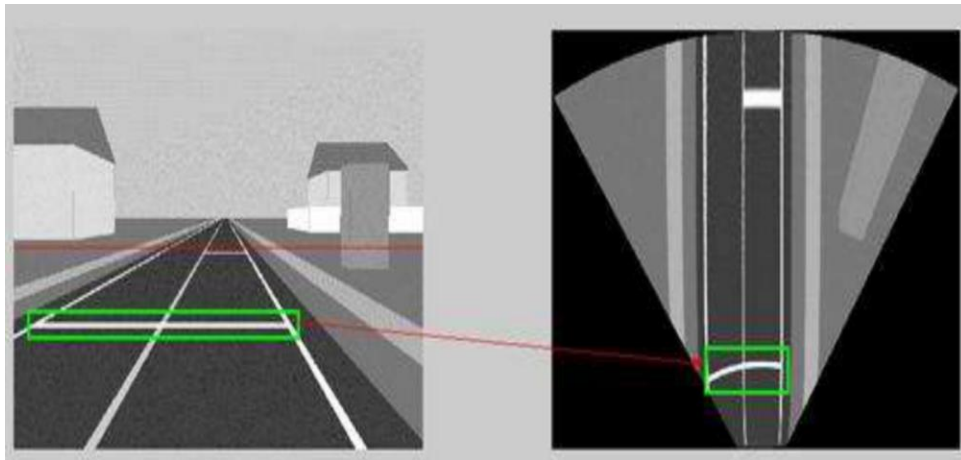
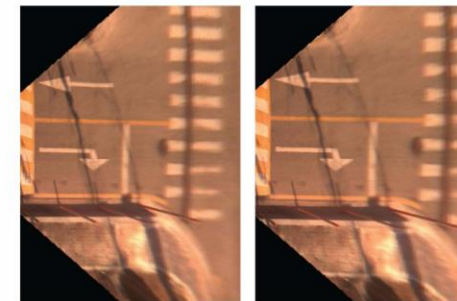
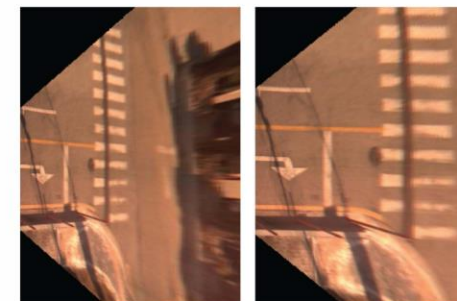


Fig. 5. Feature points derived from visual odometry. The motions of a camera (x, y, z, roll, pitch, yaw) can be calculated using this algorithm.



(a) IPM

(b) adaptive IPM



(c) IPM (after 7 frame)

(d) adaptive IPM (after 7 frame)

Fig. 6. Experimental results of the data set 1 having a speed bump. There is about 4.1 degrees of difference in the pitch angle between image (a) and (c). (c) and (d) are 7 frames after (a) and (b).

BEV Generation Methods

Weaknesses of Geometric Projection:

- Needs accurate depth or calibration
- Hard with monocular vision

2) Deep Learning-Based BEV Generation

Lift-Splat-Shoot (LSS):

To explicitly model 3D space by estimating a depth distribution for every pixel in every camera image before projecting them into a BEV grid.

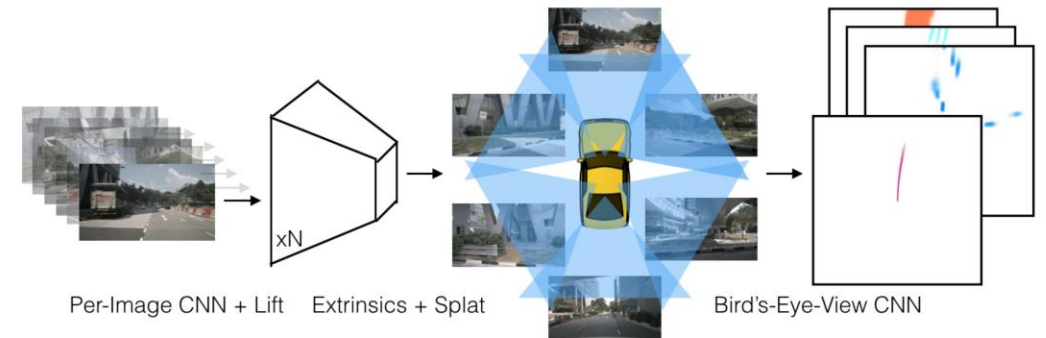
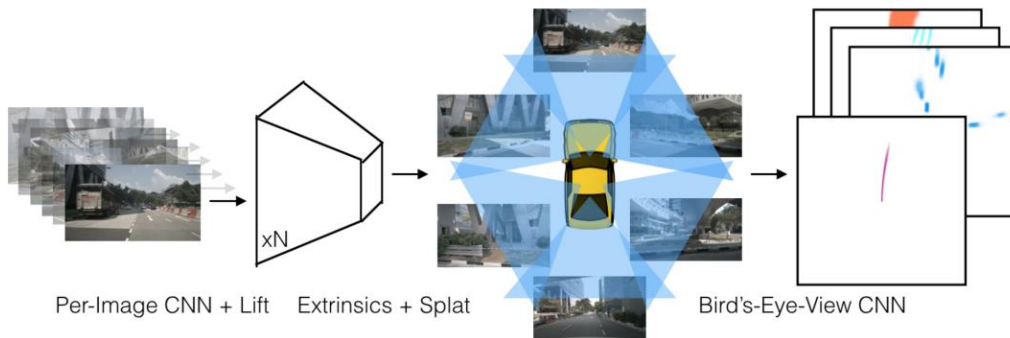


Fig. 4: **Lift-Splat-Shoot Outline** Our model takes as input n images (left) and their

BEV Generation Methods

Process of **Lift-Splat-Shoot (LSS)**:

- **Lift (3D Feature Cloud Creation)**: For each pixel in each camera image, the network doesn't choose a single depth but predicts a depth probability distribution (e.g., over discrete depth bins). This creates a "cloud" of 3D points for each image, where each point has a feature vector and a probability of existing at a certain location.
- **Splat (Voxel Pooling)**: This 3D point cloud is then projected into a predefined 3D voxel grid (a 3D grid over the car's surroundings). Features from all points that fall into the same voxel are pooled (summed/averaged) together. This step effectively "splats" the points into the grid.
- **Shoot (BEV Map Creation & Downstream Tasks)**: The 3D voxel grid is then collapsed along the height dimension (e.g., by taking the maximum value in each column) to create a 2D BEV feature map. This rich BEV map can then be used to "shoot" (perform) downstream tasks like perception, prediction, and planning.



Lift pixels → 3D → BEV projection

Fig. 4: **Lift-Splat-Shoot Outline** Our model takes as input n images (left) and their

BEV Generation Methods

2) Deep Learning-Based BEV Generation

BEVFormer:

To bypass explicit 3D reconstruction and use a transformer architecture to let the model learn how to generate a BEV representation directly from multi-camera inputs, leveraging spatial and temporal information.

- Transformer queries BEV grid
- Temporal fusion (multi-frames)

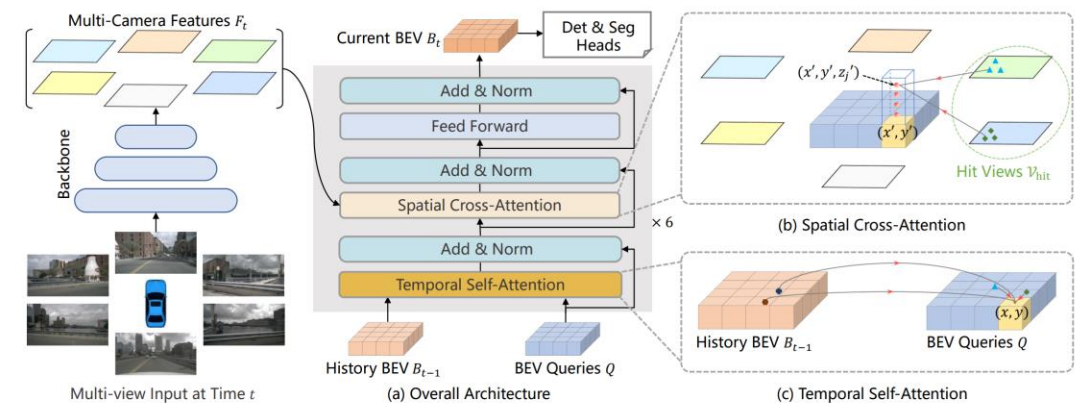


Figure 2: Overall architecture of BEVFormer. (a) The encoder layer of BEVFormer contains

BEV Generation Methods

Process of BEVFormer:

- **Define a BEV Query Grid:** The model initializes a grid of learned BEV queries. Each query is a feature vector responsible for a specific physical region in the BEV space (e.g., a 0.5m x 0.5m area).
- **Spatial Cross-Attention:** Each BEV query “looks” back into the multi-camera images to find relevant features. It projects itself into the views of all cameras and uses attention mechanisms to gather and fuse features from the image regions it needs. This is a learned projection, not a fixed geometric one.
- **Temporal Self-Attention:** This is a major advantage over LSS. Each BEV query can also interact with the BEV features from previous timesteps (e.g., the last few seconds). This allows the model to reason about motion, velocity, and occlusion by seeing how the scene has changed over time, dramatically improving perception.
- **Output:** The output of the transformer is a powerful BEV feature map that inherently contains rich spatial and temporal information, ready for task-specific heads (detection, segmentation, etc.).

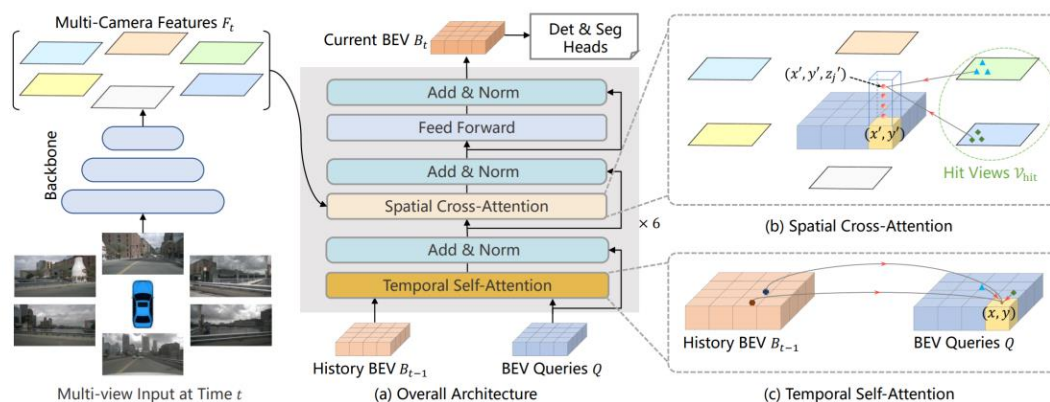
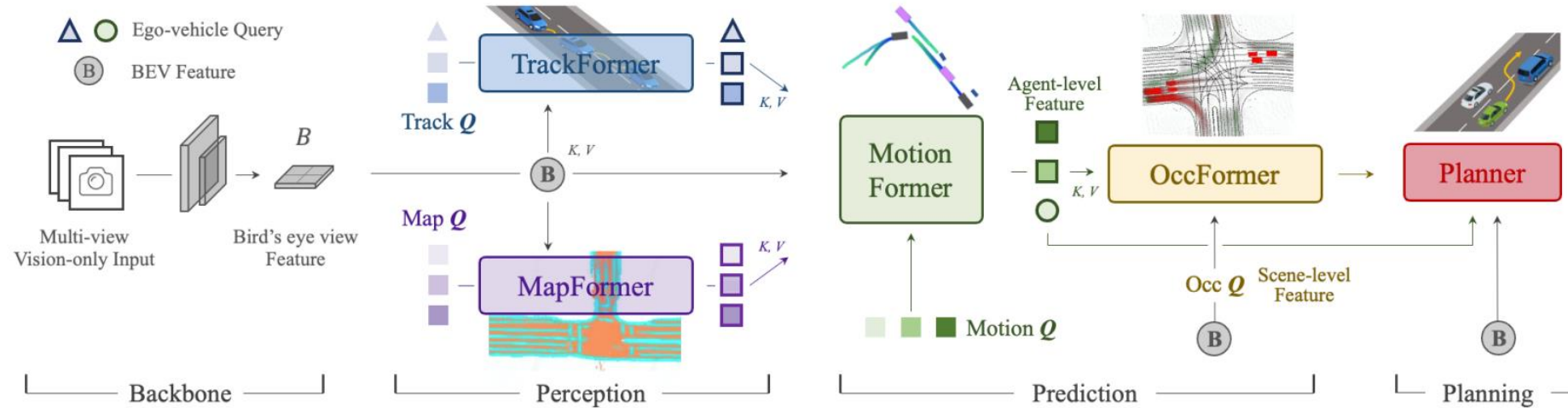


Figure 2: Overall architecture of BEVFormer. (a) The encoder layer of BEVFormer contains

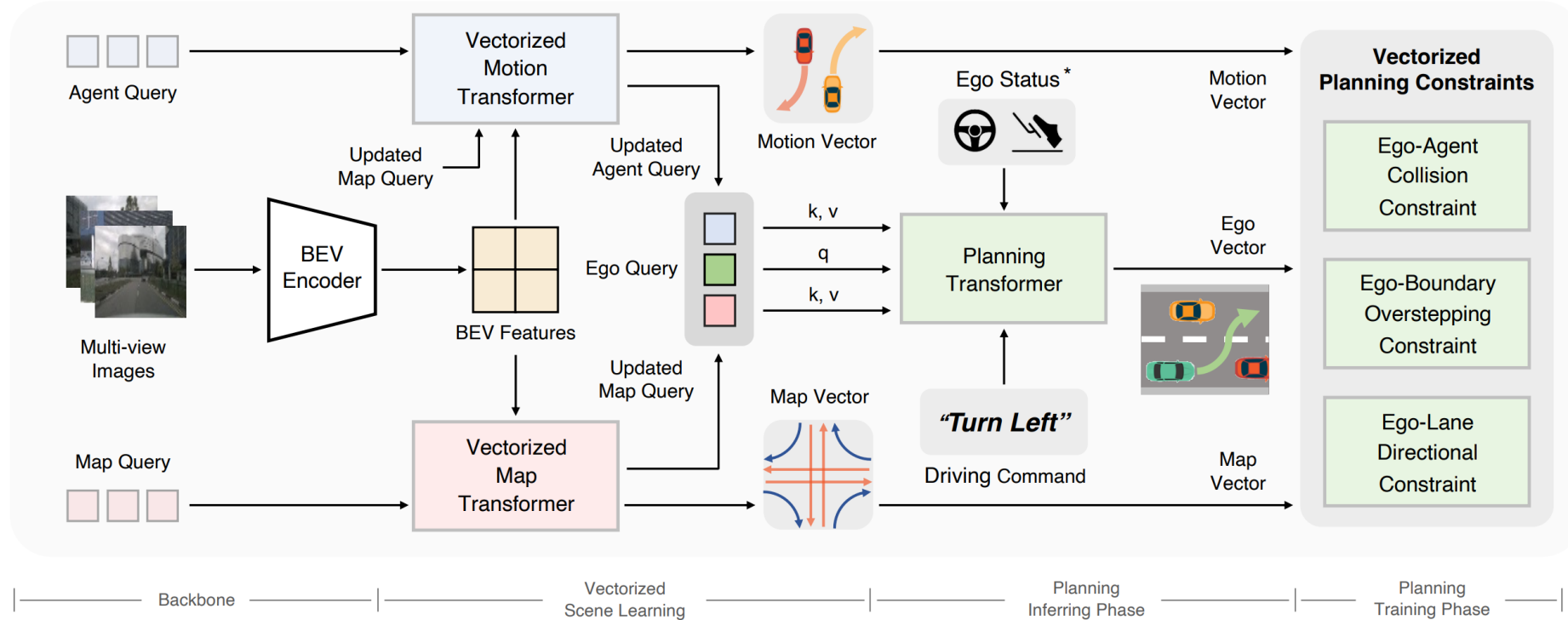
BEV-Inspired End-to-End Driving

Case 1: UniAD



BEV-Inspired End-to-End Driving

Case 2: VAD



Benchmark and Evaluation Metrics

Benchmark Datasets

Self-Driving

- Waymo Open Dataset
- nuScenes
- Argoverse 2
- KITTI BEV tasks

Indoor Navigation

- Habitat
- Matterport3D
- Gibson

Evaluation Metrics

Task	Metrics
Driving	collision rate, lane accuracy
BEV	mIoU, minFDE, ADE
Navigation	SPL, success rate

Discussion

Open Questions:

- 1) Should we trust fully end-to-end driving?
- 2) Does BEV replace maps?
- 3) Can robots learn spatial priors without geometry?

Summary

- End-to-End Navigation
 - Direct sensor input → model → control
 - Advantages: Simplified pipeline, data-driven adaptability, reduced manual work.
- BEV Representation
 - Top-down view from multi-sensor data (Camera, LiDAR, Radar).
 - Methods: Geometric projection, deep learning approaches (Lift-Splat-Shoot, BEVFormer).
 - Benefits:
 - Consistent spatial representation,
 - multi-sensor fusion,
 - improved planning interpretability.



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)